

Mini-Project Part 1

The Matrix-Multiply Unit

Building a big matrix multiply out of a small one

Three BSV designs: combinational, K-serial, double-buffered

SUBMITTED BY

Phalgun (MS 1)

ROLL NUMBER

2026702011

COURSE

CS2.501 Advanced Computer Architecture · Monsoon 2026

INSTITUTE

IIIT Hyderabad

1 Overview and measurements

This report covers three BSV implementations of the same unit, $C[4][4] += A[4][4] \times B[4][4]$, with Int8 operands and an Int32 accumulator that stays inside the unit between requests. **Design A** is fully combinational: all 64 multiply-accumulates occur in one rule firing ($\ell = 1$). **Design B** is K-serial: one outer product per cycle over four cycles ($\ell = 4$), using 16 multipliers instead of 64. **Design C** is Design B with two banks of A and B used alternately, so the next tile's loads overlap the running Mul. All three pass both workloads with **0 errors**, which is the absolute precondition for every number below.

W1 is $64 \times 64 \times 64$ (40,960 requests, 262,144 MACs, readout every 16 Muls). W2 is $64 \times 512 \times 64$ (299,008 requests, 2,097,152 MACs, readout every 128 Muls). Both run 256 output tiles through the same nine-request tile pattern: eight loads and one Mul.

1.1 Correctness and performance (from recorded test runs)

Metric	Design A	Design B	Design C
Architecture	Fully parallel	K-serial, 16 multipliers	K-serial + double buffering
W1 requests	40,960	40,960	40,960
W1 cycles	40,961	57,345	41,729
W1 stalls	1	16,385	769
W1 errors	0	0	0
W2 requests	299,008	299,008	299,008
W2 cycles	299,009	430,081	299,777
W2 stalls	1	131,073	769
W2 errors	0	0	0

Stalls = cycles – requests. Bold shaded cells mark the fewest cycles in each workload.

1.2 Hardware synthesis (make depth)

Metric	Design A	Design B	Design C
Cells (area proxy)	113,160	16,288	29,874
Logic depth (clock-period proxy)	71	59	61
Area × delay = cells × depth	8,034,360	960,992	1,822,314

Logic depth counts gate levels on the longest combinational path. It is a proxy for the clock period, deterministic and good for comparing my own designs, but it ignores gate and wire delay, so it is never quoted in nanoseconds.

1.3 Derived metrics

Design	W1 cycles × depth	W2 cycles × depth	Cells × depth	MACs / request (W1 / W2)
A (combinational)	2,908,231	21,229,639	8,034,360	6.40 / 7.01
B (K-serial)	3,383,355	25,374,779	960,992	6.40 / 7.01
C (K-serial + double buffer)	2,545,469	18,286,397	1,822,314	6.40 / 7.01

Shaded bold = best in column. MACs per request: $262,144 / 40,960 = 6.40$ (W1) and $2,097,152 / 299,008 = 7.01$ (W2).

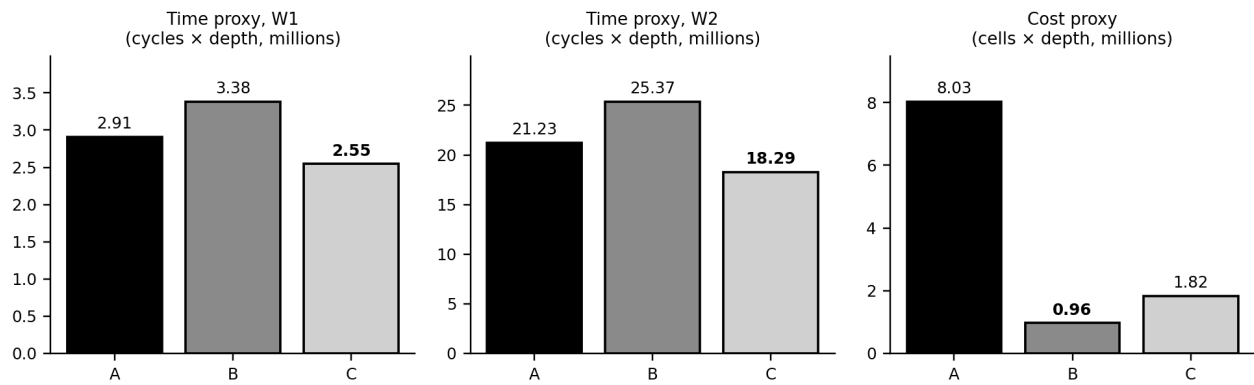


Figure 1. Time proxy (lower is better) and cost proxy for the three designs. Darkest bar = A, mid-grey = B, light = C.

The ranking changes by column, which is the trade-off the project maps. Fewest cycles: A (C is only 1.9% behind on W1). Best time (cycles × depth): C, because its depth is 10 levels below A. Smallest hardware and best cells × depth: B. No design wins every column.

2 Predictions made before building, and where they were wrong

Before measuring, I derived cycles per tile as a formula in ℓ , where ℓ is the number of cycles a Mul occupies the arithmetic. For a design where the next tile's LoadA must wait for the running Mul to release A, I predicted $8 + \ell$ cycles per tile. For a design where loads need not wait, I predicted 9. Comparing against the measured results:

Item	Predicted	Measured	Why it differed
A, cycles per tile	9	9 (+16 reads per output tile)	Held. $\ell = 1$, so the Mul finishes in the cycle after it is accepted.
B, cycles per tile	$8 + \ell = 12$	13 (4 stalls per Mul)	The Mul request is accepted in its own cycle and the $\ell = 4$ slices run <i>after</i> it. The correct cost is $9 + \ell$, not $8 + \ell$. I forgot to count the Mul's own request slot.
C, cycles per tile	9, one stall at the end	9, but 769 stalls	Loads never stall, but ReadC must wait for the last Mul of each output tile: $3 \times 256 + 1 = 769$.
Cell ranking	$B < C < A$	$B < C < A$	Held, but C is +13.6k cells over B, far more than the 512 extra flops explain (see §3).
Depth ranking	$B < C < A$	$59 < 61 < 71$	Held, but $4\times$ fewer multipliers cut depth by only 17%.

2.1 The cycle model reproduces every measurement

Putting the per-tile cost, the 16 reads per output tile and the end-of-run wait together gives one formula:

$$\text{cycles} = 256 \times (\text{K-tiles} \times \text{tile_cycles} + 16 + \text{wait}) + 1$$

Case	Substitution	Result	Measured
A, W1	$256 \times (16 \times 9 + 16 + 0) + 1$	40,961	40,961
A, W2	$256 \times (128 \times 9 + 16 + 0) + 1$	299,009	299,009
B, W1	$256 \times (16 \times 13 + 16 + 0) + 1$	57,345	57,345
B, W2	$256 \times (128 \times 13 + 16 + 0) + 1$	430,081	430,081
C, W1	$256 \times (16 \times 9 + 16 + 3) + 1$	41,729	41,729
C, W2	$256 \times (128 \times 9 + 16 + 3) + 1$	299,777	299,777

All six measured cycle counts are reproduced exactly, so the stall analysis in §3 accounts for every cycle. There is no unexplained residue.

2.2 Why the depth prediction is the instructive one

The ordering of depth was right, but the size of the effect was not what a naive count suggests. Moving from 64 multipliers to 16 removes three of the four stages of the K-sum, yet depth falls only from 71 to 59. This is the central lesson of the synthesis results: **hardware reduction and critical-path reduction are different properties**. §3 develops this.

3 Answers to the §10 questions

3.1 Where do the cycles go?

Design A has a single stall: the last response lands one cycle after the last request, and nothing else ever blocks. **Design B** has 4 stalls per Mul. While a Mul runs, every request (loads, ReadC and the next Mul) is held, so each of the 4,096 Muls in W1 costs $\ell = 4$ extra cycles: $16,384 + 1 = 16,385$ on W1 and $4 \times 32,768 + 1 = 131,073$ on W2. **Design C** has 3 stalls per output tile. Loads go into the load bank without waiting, so the only request that waits is the first ReadC of a tile, which needs the last Mul to finish: $3 \times 256 + 1 = 769$ on both workloads. Because the number of output tiles is the same in W1 and W2, C's stall count does not grow with accumulation depth.

3.2 Why is MACs per request the same for all three designs?

Because it is a property of the interface and the tile shape, not of the hardware inside. Every request carries at most one 32-bit word. One tile always needs eight loads and one Mul to perform 64 MACs, and the readout adds a further 16 requests per output tile. So the number of requests for a given amount of arithmetic is fixed before any design decision is made: $262,144 / 40,960 = 6.40$ on W1, and $2,097,152 / 299,008 = 7.01$ on W2 (the nominal $64/9 \approx 7.1$ minus the readout overhead). What the three designs change is *when* those requests are accepted, which affects cycles and stalls but never the request count.

What would have to change. Each request must carry more work. Two routes: (1) **descriptor/DMA tile loads**, where the unit is given an address and a stride and fetches the data itself, which turns nine requests per tile into three (≈ 21 MACs per request); or (2) **a larger tile**, which raises MACs per request roughly as 25. Faster internals cannot move this number.

3.3 What sets the longest path?

Each output element is a multiply followed by a sum over K, and that expression sits in series on the critical path. In Design A it is a multiplier followed by a four-input adder tree and the accumulate add. The multiplier dominates: going from A to B removes three of the four K-sum stages and 48 of the 64 multipliers, yet depth falls only from 71 to 59, about 12 levels. B also *adds* a 4:1 mux on the slice index, which eats part of what was saved. Cells fall about $7\times$ ($113,160 \rightarrow 16,288$) while depth falls only 17%.

This is the trap the assignment warns about: **“fewer multipliers” and “shorter critical path” are different properties**. Replicating a multiplier in parallel costs area but not depth, because the copies work side by side. Removing copies saves area but leaves each remaining multiplier exactly as deep as before. Only removing stages in *series* shortens the path.

Two caveats on my own numbers. First, part of the cell drop comes from B and C using signedMul ($8\times 8 \rightarrow 16$ -bit) instead of the 32-bit multipliers in A, and I did not separate that effect from the reduction in multiplier count. Second, C is 2 levels deeper than B with the same arithmetic array; the likely cause is the extra bank copy and the additional writers on the C register, but I did not verify this.

3.4 What happens when a LoadA arrives while a Mul is in progress?

§3 requires responses identical to a unit that handled every request completely, one at a time. In Design B the Mul reads all of A and B on every slice, and a LoadA rewrites a whole row of A. If it were accepted mid-Mul, slices 2 to 4 would see the new row and the product would differ from the sequential result. There is more than one legal answer, with different costs:

Design	Policy	Cost
B	Hold every request (loads, ReadC and the next Mul) while the Mul is busy. Simple, one busy flag.	$\ell = 4$ cycles per Mul; 4 stalls per Mul.
C	Copy A and B into a working bank when the Mul is accepted. Loads then fill the other bank freely. Still hold the next Mul (one engine, one C) and ReadC (C is half-accumulated).	512 more flops plus a bank bit; $\ell - 1 = 3$ cycles per <i>output tile</i> instead of ℓ per Mul.

ReadC must be held in both designs for the same reason a load is held in B: reading half-accumulated C would expose an intermediate value that the sequential reference never shows. The three rows of §3.1 show how large the difference is: C removes the per-Mul penalty entirely and keeps only a per-tile one.

3.5 Why does W2 cost more than W1 per unit of readout, but less per MAC?

Both workloads have 4,096 ReadC requests, but W1 averages $40,960 / 4,096 = \mathbf{10.0}$ cycles per read while W2 averages $299,008 / 4,096 = \mathbf{73.0}$. W2 costs more per unit of readout simply because it does 8× more work between readouts. Per MAC it is cheaper because the fixed readout cost is spread over more arithmetic: readout is 10% of requests in W1 but only 1.4% in W2.

Cycles per MAC	Design A	Design B	Design C
W1	0.1563	0.2188	0.1592
W2	0.1426	0.2051	0.1430
Drop W1 → W2	8.8%	6.2%	10.2%

Most sensitive to the workload: C. Its only overhead is per output tile (readout plus the 3-cycle wait), and deeper accumulation amortises exactly that. **Least sensitive: B.** Its 4 stalls recur on every Mul and never amortise, so extending K only dilutes the (smaller) readout share. A sits in between, with readout as its only overhead beyond the end stall.

3.6 How wide does the accumulator have to be?

The largest single product is $(-128) \times (-128) = 2^{14}$, so $|\text{product}| \leq 2^{2 \cdot W_{\text{ELEM}} - 2}$. With K products per Mul and N Muls between readouts, $|C| \leq N \cdot K \cdot 2^{2 \cdot W_{\text{ELEM}} - 2}$. A signed accumulator holds up to $2^{W_{\text{ACC}} - 1} - 1$, which gives

$$W_{\text{ACC}} \geq 2 \cdot W_{\text{ELEM}} + \lceil \log_2(K \cdot N) \rceil$$

For $W_{\text{ELEM}} = 8$ and $K = 4$ this is $18 + \lceil \log_2 N \rceil$. At $W_{\text{ACC}} = \mathbf{32}$, $N \leq 2^{14} = 16,384$ Muls between readouts (W2 needs only 128, so there is ample margin). At $W_{\text{ACC}} = \mathbf{16}$ the bound yields $K \cdot N < 1$: a single Mul can already overflow in the worst case ($4 \times 16,384 = 65,536 > 32,767$), and even a single product plus one more can reach 2^{15} . So no number of Muls is guaranteed safe. This is why accumulators are wider than operands, and why the readout-and-zero structure exists.

3.7 When does a DMA-fed unit stop being instruction-bound?

For an $S \times S \times S$ tile, data moved is $2S^2$ bytes and work is S^3 MACs, so **MACs per byte** = $S/2$. Instructions per tile fall to about 3 (two tile loads and a Mul), so **MACs per instruction** $\approx \mathbf{2S}$. The two ratios differ by exactly 4, the number of Int8 elements in a 32-bit word. DMA does not reduce the data that must move; it reduces how many instructions it takes to name it.

For the $4 \times 4 \times 4$ unit: 16 bytes of A + 16 bytes of B for 64 MACs is 2 MACs per byte. Assuming 64 MACs per cycle at 1 GHz (Appendix A), the unit needs $64 / 2 = \mathbf{32 \text{ bytes per cycle} = 32 \text{ GB/s}}$, more than a single DRAM channel supplies. In general the requirement is $128 \cdot f / S$ bytes per second. So a DMA-fed $4 \times 4 \times 4$ unit becomes memory-bound as soon as instruction supply stops limiting it; with a 16 GB/s budget, for example, the tile must be at least $S = 8$.

One guarantee the OS must provide that §2 does not need: page pinning. The pages must not move or be swapped out while a transfer is in flight, because the DMA engine addresses memory independently of the CPU. (Cache coherence and address translation are the other two requirements. The §2 interface needs none of them because every byte moves through a register the CPU already owns.)

3.8 When BSV does not do what you expect: make schedule

bsc schedules whole rules, so the design has one rule per request type. In Designs B and C, the step rule, the Mul rule and ReadC all write the C register. They are made mutually exclusive by the `rg_busy` guard rather than by rule ordering, and in B the loads are guarded by it too. Using an explicit guard keeps the conflict report clean and makes the sequential-order requirement of §3 visible in the code instead of depending on scheduler urgency.

4 Design choice

Chosen: Design C, under the priority **minimum time to finish the job (cycles × logic depth) at moderate area**.

C is best on that measure on both workloads: 2,545,469 on W1 and 18,286,397 on W2, which is 14% lower than A on W2 (21,229,639) and well below B (25,374,779). It does this while being 3.8× smaller than A (29,874 vs 113,160 cells). It wins even though A has fewer cycles, because C's logic depth is 10 levels lower, which in a real design would translate into a shorter clock period. The cost over B is 512 extra flops of state plus a bank-select bit, and the small per-tile ReadC wait.

Strongest argument against this choice

Under an area-first priority, B is the right design. B has 1.9× better cells × depth (960,992 vs 1,822,314) with almost half the cells. Further, C's advantage is the cycles it hides per Mul, and that only matters if the requester issues a request every cycle, as the testbench does for free. In Part 2 a real in-order CPU may not (§12): it must fetch, decode and execute a load for every word, so request gaps may already be wider than B's $l = 4$ stall. In that case C's cycle gain over B would shrink while its area cost stays. I would revisit the choice once Part 2 measures the real issue rate.

Limits of these results

Logic depth is a gate-level proxy, not static timing analysis, so the cycles × depth comparison assumes clock period scales with depth. Two effects are unseparated or unverified in my data: the contribution of signedMul to B and C's cell reduction, and the cause of C's 2-level depth penalty relative to B. Neither changes the ranking, but both would sharpen the area explanation.